

Documentação Técnica

Site Institucional para Departamentos (Template Laravel)

Projeto: departamento-ufop

Agosto de 2026

Resumo. Este documento descreve a arquitetura, o funcionamento e os componentes de código de um site institucional desenvolvido para departamentos (modelo inspirado nos portais da Universidade Federal de Ouro Preto — UFOP), construído em Laravel, servido via Docker e projetado deliberadamente **sem banco de dados**: todo o conteúdo editável é armazenado em arquivos de texto (JSON) no próprio disco do servidor. O objetivo é oferecer um site rápido de implantar, fácil de manter e com uma superfície de ataque reduzida em relação a soluções tradicionais baseadas em CMS de terceiros (como WordPress).

Sumário

1	Introdução	3
1.1	Tecnologias utilizadas	3
2	Visão Geral: Como o Sistema Funciona	3
2.1	Não existe banco de dados	3
2.2	O ciclo de vida de uma requisição	4
2.3	O painel administrativo, sem tabela de usuários	4
2.4	Conteúdo dinâmico vs. estático	5
3	Estrutura de Pastas do Projeto	5
4	Documentação de Código	6
4.1	Rotas (routes/web.php)	6
4.1.1	Rotas públicas	6
4.1.2	Rotas do painel administrativo (prefixo /admin)	6
4.2	Controllers	7
4.2.1	SiteController	7
4.2.2	NoticiaController e EventoController (públicos)	7
4.2.3	Admin\AuthController	7
4.2.4	Admin\ContentController	7
4.2.5	Admin\NoticiaController e Admin\EventoController	7
4.3	Middlewares	7
4.4	Classes de suporte (app/Support)	8
4.4.1	ContentStore	8
4.4.2	ContentDefaults	8
4.4.3	NoticiaStore e EventoStore	8
4.4.4	ImageUploader	9
4.5	Provider (AppServiceProvider)	9
4.6	Configuração	9
4.7	Views (Blade)	9
4.8	Esquema de conteúdo por seção	10
5	Identidade Visual	10
6	Segurança	11
6.1	Proteções estruturais (decorrentes da arquitetura)	11
6.2	Proteções adicionadas explicitamente	11
6.3	Limitações conhecidas (honestidade técnica)	12
7	Docker e Implantação (Deploy)	12
8	Reaproveitando o Template para Outro Departamento	12
9	Referências	13

1 Introdução

Este projeto nasceu da necessidade de um departamento acadêmico (no molde dos institutos e departamentos da UFOP) ter um site institucional próprio, com identidade visual da instituição, que pudesse ser editado no dia a dia por uma secretária ou servidor **sem qualquer conhecimento de programação**, e que ao mesmo tempo fosse mais simples e mais seguro de operar do que uma instalação WordPress tradicional — plataforma que, nos servidores da própria UFOP, historicamente sofre ataques com frequência (em geral por falhas em plugins de terceiros).

A decisão de projeto mais importante, tomada logo no início, foi: **o site não usa banco de dados**. Em vez de MySQL, PostgreSQL ou até SQLite, todo o conteúdo (textos, listas de notícias, membros de equipe, configurações) é guardado em arquivos `.json` dentro da pasta `storage/` do próprio Laravel. As implicações dessa escolha — vantagens de simplicidade e segurança, e as poucas desvantagens — são discutidas ao longo deste documento.

1.1 Tecnologias utilizadas

- **Backend:** PHP 8.4 + Laravel 13 (framework), sem Eloquent/ORM para o conteúdo do site.
- **Frontend:** Blade (motor de templates do Laravel) sobre o tema visual *Neoton* (HTML/CSS/JS adquirido comercialmente), adaptado e re-corado com a paleta institucional da UFOP.
- **Servidor web:** Apache 2.4 (imagem oficial `php:8.4-apache`).
- **Contêineres:** Docker + Docker Compose (um único serviço, sem banco de dados).
- **Persistência:** sistema de arquivos, com dois diretórios montados como volume Docker para sobreviver a reconstruções da imagem: `storage/content/` (JSON) e `storage/uploads/` (imagens/anexos enviados pelo painel).

2 Visão Geral: Como o Sistema Funciona

Esta seção explica, em linhas gerais e sem entrar em detalhes de código, o que acontece "por trás das cortinas" quando alguém visita o site ou quando a secretária edita alguma informação pelo painel administrativo.

2.1 Não existe banco de dados

Um site tradicional (WordPress, Joomla, e a grande maioria dos sistemas administráveis) guarda o conteúdo dentro de um Sistema Gerenciador de Banco de Dados (SGBD), como MySQL ou PostgreSQL: um programa separado, rodando ao lado da aplicação, que armazena os dados em tabelas e responde a consultas na linguagem SQL.

Neste projeto, essa peça inteira foi removida. Em vez de tabelas, cada "seção" do site (a página inicial, a lista de notícias, a equipe, etc.) tem um arquivo de texto simples no formato JSON, guardado em `storage/app/private/content/`. Por exemplo, o arquivo `noticias.json` contém, de forma legível por humanos, todas as notícias e editais já publicados:

Listing 1: Exemplo real de `storage/content/noticias.json`

```
1 {
2   "items": [
3     {
4       "id": "exemplo-noticia",
5       "tipo": "noticia",
6       "titulo": "Departamento realiza evento de boas-vindas",
7       "resumo": "Confira como foi o evento...",
```

```
8         "conteudo": "Texto completo da noticia...",
9         "imagem": "assets/images/hero-blog/1.jpg",
10        "anexo": "",
11        "data_publicacao": "2026-01-15"
12    }
13 ]
14 }
```

Quando um visitante acessa `/noticias`, o Laravel simplesmente lê esse arquivo, decodifica o JSON para um array PHP, ordena pelas datas e desenha a página HTML em cima desses dados. Não existe nenhuma consulta SQL em lugar nenhum do código da aplicação.

Vantagem direta de segurança: como não existe banco de dados nem linguagem de consulta sendo interpretada, o site é estruturalmente imune a *SQL Injection* — uma das duas classes de ataque mais comuns contra sites WordPress.

2.2 O ciclo de vida de uma requisição

Quando alguém digita o endereço do site no navegador, acontece, resumidamente, o seguinte percurso dentro da aplicação Laravel:

1. O Apache recebe a requisição HTTP e a repassa ao PHP (`public/index.php`), ponto de entrada único do Laravel.
2. O roteador (`routes/web.php`) identifica qual URL foi acessada (por exemplo, `/noticias`) e qual *controller* deve tratá-la.
3. O **controller** (por exemplo, `NoticiaController`) busca os dados necessários chamando uma classe de suporte (`NoticiaStore`), que por sua vez lê o arquivo JSON correspondente do disco através da classe genérica `ContentStore`.
4. O controller entrega esses dados para uma **view** Blade (um arquivo `.blade.php`), que combina o HTML do tema com os dados reais e produz a página final.
5. Antes de a resposta sair, um conjunto de **middlewares** (camadas que interceptam a requisição/resposta) adicionam cabeçalhos de segurança e, no caso das rotas do painel, verificam se o usuário está autenticado e registram a ação num log de auditoria.

2.3 O painel administrativo, sem tabela de usuários

Diferente da maioria dos sistemas, este projeto não tem uma tabela `users` nem um sistema de cadastro — existe **uma única conta de administrador**, cujo e-mail e senha (já criptografada com *bcrypt*) ficam guardados em variáveis de ambiente (`.env`), nunca em texto puro e nunca no código-fonte.

O login funciona da seguinte forma:

1. A secretária acessa `/admin/login` e informa e-mail e senha.
2. O `Admin\AuthController` compara o e-mail com `ADMIN_EMAIL` e verifica a senha com `Hash::check()` contra o hash `ADMIN_PASSWORD_HASH`.
3. Se corretos, a sessão do navegador recebe uma marca (`admin_logged_in = true`) — é essa marca que o middleware `EnsureAdminAuthenticated` verifica em toda rota protegida do painel.
4. Opcionalmente, se a caixa "Lembrar por 30 dias" estiver marcada, um cookie adicional, criptografado pelo próprio Laravel e amarrado ao hash de senha atual, mantém a sessão ativa mesmo depois de fechar o navegador (ver Seção 6).

2.4 Conteúdo dinâmico vs. estático

É importante distinguir dois tipos de "seção" no sistema:

- **Seções de página única** (Configurações, Rodapé, Página Inicial, Sobre, Serviços, Graduação, Pós-Graduação, Equipe, Contato, Carrossel): cada uma tem *um único* arquivo JSON, editado através de *um único* formulário no painel — a secretária altera os campos e salva tudo de uma vez.
- **Seções de lista com CRUD completo** (Notícias/Editais e Eventos): cada publicação é um item independente, com seu próprio identificador, e o painel oferece telas de *listar*, *criar*, *editar* e *excluir* — muito mais parecido com o funcionamento de um blog tradicional, mas ainda gravando tudo num único arquivo JSON por trás (um array de itens).

3 Estrutura de Pastas do Projeto

A árvore abaixo mostra as pastas e arquivos mais relevantes dentro de **site/** (raiz do projeto Laravel e do repositório Git):

Listing 2: Estrutura simplificada de pastas

```

1  site/
2  |-- app/
3  |   |-- Http/
4  |   |   |-- Controllers/
5  |   |   |   |-- SiteController.php
6  |   |   |   |-- NoticiaController.php
7  |   |   |   |-- EventoController.php
8  |   |   |-- Admin/
9  |   |   |   |-- AuthController.php
10 |   |   |   |-- ContentController.php
11 |   |   |   |-- NoticiaController.php
12 |   |   |   |-- EventoController.php
13 |   |   |-- Middleware/
14 |   |   |   |-- EnsureAdminAuthenticated.php
15 |   |   |   |-- SecurityHeaders.php
16 |   |   |   |-- AdminAuditLog.php
17 |   |   |-- EnsureHttps.php
18 |   |-- Providers/AppServiceProvider.php
19 |   |-- Support/
20 |   |   |-- ContentStore.php
21 |   |   |-- ContentDefaults.php
22 |   |   |-- NoticiaStore.php
23 |   |   |-- EventoStore.php
24 |   |   |-- ImageUploader.php
25 |-- config/
26 |   |-- admin.php
27 |   |-- logging.php
28 |-- resources/views/
29 |   |-- layouts/app.blade.php
30 |   |-- partials/ (header, footer, breadcrumb)
31 |   |-- site/ (paginas publicas)
32 |   |-- admin/ (paginas do painel)
33 |-- routes/web.php
34 |-- public/assets/ (CSS, JS, fontes e imagens do tema)
35 |-- storage/
36 |   |-- content/ (JSON - conteudo do site, fora do Git)
37 |   |-- uploads/ (imagens/anexos enviados, fora do Git)
38 |-- DOC/ (esta documentacao)
39 |-- Dockerfile
40 |-- docker-compose.yml

```

4 Documentação de Código

Esta seção descreve, componente por componente, tudo o que foi desenvolvido especificamente para este projeto (o *framework* Laravel em si não é documentado aqui, apenas o código de aplicação escrito sob medida).

4.1 Rotas (routes/web.php)

4.1.1 Rotas públicas

URL	Nome da rota	Descrição
/	home	Página inicial (carrossel, destaques, últimas notícias, "quem somos")
/sobre	sobre	Sobre o Departamento
/servicos	servicos	Lista de serviços oferecidos
/graduacao	graduacao	Cursos de graduação
/pos-graduacao	pos-graduacao	Programas de pós-graduação
/equipe	equipe	Membros da equipe
/contato	contato	Informações de contato e mapa
/noticias	noticias.index	Listagem de notícias/editais, com filtro e paginação
/noticias/{id}	noticias.show	Detalhe de uma notícia/edital
/eventos	eventos.index	Listagem de eventos (quando ativada)

4.1.2 Rotas do painel administrativo (prefixo /admin)

Todas as rotas abaixo, exceto login, exigem os middlewares `admin.auth` (autenticação) e `admin.audit` (log de auditoria).

URL	Método	Descrição
/admin/login	GET/POST	Tela e processamento de login (com limite de tentativas)
/admin/logout	POST	Encerra a sessão
/admin	GET	Painel principal (lista todas as seções)
/admin/configuracoes	GET/POST	Nome do site, logo do cabeçalho, redes sociais
/admin/rodape	GET/POST	Logo do rodapé, texto, contato exibido, direitos autorais
/admin/home	GET/POST	Título/subtítulo da home e cards de destaque
/admin/carrossel	GET/POST	Imagens do carrossel da home
/admin/sobre	GET/POST	Texto e imagem da página Sobre
/admin/servicos	GET/POST	Lista de serviços
/admin/graduacao	GET/POST	Cursos + visibilidade no menu
/admin/pos-graduacao	GET/POST	Programas + visibilidade no menu
/admin/equipe	GET/POST	Membros da equipe (com foto)
/admin/contato	GET/POST	Dados da página de contato
/admin/noticias	GET	Lista notícias/editais
/admin/noticias/criar	GET/POST	Criar publicação
/admin/noticias/{id}/editar	GET/PUT	Editar publicação

URL	Método	Descrição
/admin/noticias/{id}	DELETE	Excluir publicação
/admin/eventos	GET	Lista eventos + interruptor de visibilidade no menu
/admin/eventos/visibilidade	POST	Liga/desliga "Eventos" no menu e na home
/admin/eventos/criar	GET/POST	Criar evento
/admin/eventos/{id}/editar	GET/PUT	Editar evento
/admin/eventos/{id}	DELETE	Excluir evento

4.2 Controllers

4.2.1 SiteController

Responsável pelas páginas públicas de conteúdo único (home, sobre, serviços, graduação, pós-graduação, equipe, contato). Cada método busca o conteúdo correspondente via `ContentStore::get()`, usando `ContentDefaults` como valor padrão, e devolve a view Blade da respectiva página. O método `home()` é o mais elaborado: também consulta `NoticiaStore::latest()` (últimas notícias) e, se a seção de eventos estiver ativada, `EventoStore::upcoming()` (próximos eventos), ajustando o layout da página inicial para duas colunas nesse caso.

4.2.2 NoticiaController e EventoController (públicos)

Cuidam da listagem e exibição pública de notícias/editais e eventos. O `NoticiaController` implementa paginação manual usando `Illuminate\Pagination\LengthAwarePaginator` diretamente sobre um array PHP (já que não há consulta a banco de dados para paginar).

4.2.3 Admin\AuthController

Login, logout e a lógica do cookie "lembrar-me". Registra tentativas de login (sucesso e falha) no canal de log `admin` (Seção 6).

4.2.4 Admin\ContentController

O maior controller do projeto: concentra a edição de todas as seções de "página única" (configurações, rodapé, home, carrossel, sobre, serviços, graduação, pós-graduação, equipe, contato). Cada seção tem um par de métodos `{secao}Edit()` / `{secao}Update()`. Um método auxiliar privado, `removeLinhasVazias()`, filtra linhas vazias enviadas pelos formulários com listas repetíveis (destaques, itens de serviço). Graduação e Pós-Graduação compartilham a mesma view (`admin.sections.cursos`) e o mesmo método auxiliar `salvarPaginaCursos()`, já que têm estrutura idêntica.

4.2.5 Admin\NoticiaController e Admin\EventoController

Implementam um CRUD completo (`index`, `create`, `store`, `edit`, `update`, `destroy`) sobre suas respectivas *stores*. O de Eventos tem um método adicional, `updateVisibilidade()`, que liga/desliga o item "Eventos" do menu principal e o bloco de eventos da página inicial simultaneamente.

4.3 Middlewares

EnsureAdminAuthenticated

Protege as rotas do painel. Verifica a sessão e, na ausência dela, o cookie de "lembrar-me"; caso nenhum dos dois seja válido, redireciona para o login.

SecurityHeaders

Adiciona cabeçalhos HTTP de segurança a toda resposta (detalhado na Seção 6).

AdminAuditLog

Registra, no canal de log `admin`, toda requisição POST/PUT/PATCH/DELETE bem-sucedida feita dentro do painel — nome da rota, método, IP e e-mail do administrador.

EnsureHttps

Redireciona para HTTPS quando a variável de ambiente `FORCE_HTTPS` está ativada. Desligado por padrão.

4.4 Classes de suporte (app/Support)

4.4.1 ContentStore

O alicerce de todo o sistema de conteúdo. Duas funções estáticas apenas:

Listing 3: app/Support/ContentStore.php (resumido)

```
1 class ContentStore
2 {
3     public static function get(string $key, array $defaults = []): array
4     {
5         $disk = Storage::disk('local');
6         $path = "content/{ $key }.json";
7
8         if (! $disk->exists($path)) {
9             return $defaults;
10        }
11
12        $saved = json_decode($disk->get($path), true);
13
14        return is_array($saved) ? array_replace($defaults, $saved) :
15            $defaults;
16    }
17
18    public static function save(string $key, array $data): void
19    {
20        Storage::disk('local')->put(
21            "content/{ $key }.json",
22            json_encode($data, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT)
23        );
24    }
25 }
```

O uso de `array_replace($defaults, $saved)` garante que, mesmo que um campo novo seja adicionado ao sistema depois que o arquivo já existia (por exemplo, ao evoluir o site), o valor padrão desse campo novo é usado até que a secretária o edite pela primeira vez — não há erro por "campo faltando".

4.4.2 ContentDefaults

Uma classe com um método estático por seção (`configuracoes()`, `rodape()`, `home()`, `sobre()`, `servicos()`, `graduacao()`, `posGraduacao()`, `equipe()`, `contato()`, `carrossel()`, `noticias()`, `eventos()`), cada um devolvendo o array de valores padrão/exemplo daquela seção. É a "fonte da verdade" de como um site novo aparece antes de qualquer edição.

4.4.3 NoticiaStore e EventoStore

Implementam um pequeno "repositório" de itens sobre um único arquivo JSON, com métodos `all()`, `find($id)`, `save($item)` (cria ou atualiza, conforme a presença de um `id`) e `delete($id)`.

Os identificadores são gerados com `Str::random(8)`, verificando colisão contra os já existentes. O `EventoStore` ainda guarda, no mesmo arquivo, um sinalizador `mostrar_menu` (`mostrarMenu()` / `setMostrarMenu()`), separado da lista de itens.

4.4.4 ImageUploader

Recebe um arquivo enviado por formulário e devolve o caminho onde ele foi salvo. Dois cuidados de segurança relevantes: (1) se nenhum arquivo novo for enviado, devolve o valor atual (permite editar outros campos sem re-enviar a imagem); (2) a extensão do arquivo salvo é obtida do **conteúdo real detectado** (`$file->extension()`), não do nome que o navegador do usuário informou — evitando que um arquivo malicioso disfarçado receba uma extensão enganosa.

4.5 Provider (AppServiceProvider)

Dois papéis principais:

1. Registra um *View Composer* global (`View::composer('*', ...)`) que injeta em **toda** view renderizada as variáveis `$siteSettings`, `$rodape`, `$menuGraduacao`, `$menuPosGraduacao` e `$menuEventosVisivel` — é assim que o cabeçalho e o rodapé, presentes em todas as páginas, têm acesso aos dados de configuração sem que cada controller precise repassá-los manualmente.
2. Define o limitador de taxa (*rate limiter*) nomeado `login`, usado pela rota de autenticação (Seção 6).

4.6 Configuração

`config/admin.php`

Expõe `ADMIN_EMAIL` e `ADMIN_PASSWORD_HASH` do `.env` para o restante da aplicação.

`config/logging.php`

Define o canal `admin` (arquivo diário, `storage/logs/admin-AAAA-MM-DD.log`, retenção de 90 dias) usado pelo log de auditoria.

`config/app.php`

Adiciona a chave `force_https`, ligada à variável de ambiente `FORCE_HTTPS`.

4.7 Views (Blade)

`layouts/app.blade.php`

Layout-mãe de todas as páginas públicas: cabeçalho de `<head>` (favicon, CSS), tela de carregamento, inclusão do cabeçalho/rodapé, botão flutuante "voltar ao topo" com anel de progresso em SVG, e os scripts JS do tema.

`partials/header.blade.php`

Cabeçalho: logo, nome do site, menu principal (com os itens condicionais de Graduação/Pós-Graduação/Eventos) e ícones de redes sociais/acesso ao admin.

`partials/footer.blade.php`

Rodapé: logo, texto, lista de "links rápidos" **gerada dinamicamente** a partir dos mesmos itens do menu principal (dividida em duas colunas quando ultrapassa 5 itens), dados de contato e nota de direitos autorais.

`partials/breadcrumb.blade.php`

Faixa de título usada no topo das páginas internas (Sobre, Serviços, Notícias, etc.).

`site/*.blade.php`

Uma view por página pública.

`admin/layout.blade.php`

Layout do painel: barra superior, menu lateral com todas as seções, área de mensagens de sucesso/erro.

`admin/sections/*.blade.php`

Um formulário por seção de página única.

`admin/noticias/*.blade.php` e `admin/eventos/*.blade.php`

Telas de listagem e formulário de criação/edição do CRUD de notícias e eventos.

Listas repetíveis dentro dos formulários (destaques da home, itens de serviço, cursos, membros da equipe, slides do carrossel) seguem sempre o mesmo padrão de JavaScript: um botão "+ Adicionar" clona um `<template>` HTML, usando um **contador que nunca reaproveita um índice já usado** — mesmo depois de remover uma linha do meio da lista. Essa escolha evita um problema real encontrado durante o desenvolvimento: se dois campos diferentes acabassem sendo enviados ao servidor com o mesmo índice numérico, um sobrescreveria silenciosamente os dados do outro (por exemplo, apagando a foto de um membro da equipe ao adicionar outro).

4.8 Esquema de conteúdo por seção

A tabela a seguir resume os campos armazenados em cada arquivo JSON dentro de `storage/content/`.

Arquivo	Campos principais
<code>configuracoes.json</code>	<code>nome_site</code> , <code>sigla</code> , <code>logo</code> , <code>facebook</code> , <code>instagram</code> , <code>twitter</code> , <code>linkedin</code> , <code>youtube</code>
<code>rodape.json</code>	<code>logo</code> , <code>texto</code> , <code>copyright</code> , <code>endereco</code> , <code>telefone</code> , <code>email</code>
<code>home.json</code>	<code>hero_titulo</code> , <code>hero_subtitulo</code> , <code>destaques[]</code> (<code>icone</code> , <code>titulo</code> , <code>texto</code>), <code>sobre_titulo</code> , <code>sobre_texto</code>
<code>carrossel.json</code>	<code>slides[]</code> (<code>imagem</code> , <code>legenda</code>)
<code>sobre.json</code>	<code>titulo</code> , <code>texto_intro</code> , <code>texto_corpo</code> , <code>imagem</code> , <code>destaques[]</code>
<code>servicos.json</code>	<code>titulo</code> , <code>introducao</code> , <code>itens[]</code> (<code>icone</code> , <code>titulo</code> , <code>texto</code>)
<code>graduacao.json</code> / <code>pos_graduacao.json</code>	<code>titulo</code> , <code>texto_intro</code> , <code>cursos[]</code> (<code>nome</code> , <code>link</code>), <code>mostrar_menu</code>
<code>equipe.json</code>	<code>titulo</code> , <code>introducao</code> , <code>membros[]</code> (<code>nome</code> , <code>cargo</code> , <code>foto</code>)
<code>contato.json</code>	<code>titulo</code> , <code>texto_intro</code> , <code>endereco</code> , <code>telefone_1/2</code> , <code>email_1/2</code> , <code>mapa_embed_url</code>
<code>noticias.json</code>	<code>items[]</code> (<code>id</code> , <code>tipo</code> , <code>titulo</code> , <code>resumo</code> , <code>conteudo</code> , <code>imagem</code> , <code>anexo</code> , <code>data_publicacao</code>)
<code>eventos.json</code>	<code>items[]</code> (<code>id</code> , <code>titulo</code> , <code>data_evento</code> , <code>local</code> , <code>descricao</code> , <code>link</code>), <code>mostrar_menu</code>

5 Identidade Visual

O HTML/CSS de base é o tema comercial *Neoton* (originalmente um template de blog/revista), mantido como referência intacta em `neoton-html-full-package/` (fora do repositório Git, por questões de licença de redistribuição). Apenas os arquivos estáticos necessários (CSS, JavaScript, fontes e algumas imagens genéricas) foram copiados para `public/assets/`.

Toda a marca visual da UFOP foi aplicada através de uma única folha de estilo adicional, `public/assets/css/brand.css`, carregada por último (depois do CSS do tema), que sobrescreve

cores, tipografia de botões e o layout do cabeçalho/rodapé sem precisar alterar o tema original. As variáveis de cor centrais são:

Variável CSS	Cor	Uso
-brand-wine	#962038	Vinho institucional (marca, botões, faixa hero)
-brand-red-strong	#C8102E	Vermelho de destaque (hover, acentos)
-brand-charcoal	#2B2B2B	Grafite (cabeçalho, rodapé)
-brand-gray / -brand-gray-light	#6E6E6E / #919191	Detalhes, bordas

O logo (brasão da UFOP) foi processado a partir de uma imagem oficial de alta resolução, removendo o fundo cinza sólido (técnica de *chroma key* via script Python/Pillow), para que o brasão branco se integre ao cabeçalho escuro sem aparecer como um retângulo "colado" por cima. O favicon do site foi gerado a partir do mesmo brasão, recortado apenas na parte do símbolo (sem o texto "UFOP", que fica ilegível em tamanho de ícone de aba).

6 Segurança

6.1 Proteções estruturais (decorrentes da arquitetura)

- **Sem SQL Injection:** não existe banco de dados nem consulta SQL em nenhuma parte do código da aplicação.
- **Sem XSS armazenado via terceiros:** o projeto não instala nenhum plugin de terceiros (as únicas dependências PHP são o próprio `laravel/framework` e `laravel/tinker`). Todo conteúdo digitado pelo administrador é impresso nas views usando a sintaxe `{{ }}` do Blade, que aplica *escape* de HTML automaticamente — nenhuma view usa a sintaxe `{!! !!}` (que imprimiria HTML sem escapar) sobre conteúdo vindo do painel.
- **Superfície de ataque mínima:** não há formulário público de comentário, cadastro ou upload — a única forma de gravar conteúdo no site é autenticando-se como administrador.

6.2 Proteções adicionadas explicitamente

Limite de tentativas de login

O *rate limiter login* (`AppServiceProvider`) permite no máximo 5 tentativas por minuto, chaveadas por combinação de e-mail tentado + endereço IP, dificultando ataques de força bruta contra a conta única do administrador.

Log de auditoria

Todo login (sucesso ou falha) e toda ação que altera conteúdo ficam registrados em `storage/logs/admin-*.log` com data/hora, IP e rota afetada — essencial para investigar qualquer incidente.

Cabeçalhos HTTP de segurança

O middleware `SecurityHeaders` adiciona, a toda resposta: `X-Frame-Options` (evita que o site seja carregado dentro de um `<iframe>` de terceiros — proteção contra *clickjacking*), `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy` e uma `Content-Security-Policy` restritiva (o site não depende de nenhum CDN externo, então a política pode ser fechada em `'self'`).

Uploads de arquivo

A extensão do arquivo salvo vem do tipo de conteúdo **efetivamente detectado**, não do nome informado pelo navegador; e o Apache recusa (HTTP 403) qualquer tentativa de

executar um arquivo `.php` a partir do caminho `/storage/*`, como defesa adicional caso algum arquivo malicioso um dia burle a validação de tipo.

HTTPS opcional

A variável `FORCE_HTTPS` (desligada por padrão) ativa redirecionamento automático para HTTPS — deve ser ligada apenas depois de o domínio final ter certificado configurado.

6.3 Limitações conhecidas (honestidade técnica)

Nenhuma dessas medidas substitui um antivírus de verdade: a validação de upload confirma que um arquivo *é* do tipo que alega ser (uma imagem, um PDF, um Word), mas não varre o conteúdo em busca de assinaturas de malware conhecido. Em particular, um documento do Word/Excel com macro maliciosa pode passar pela validação — o risco nesse caso recai sobre quem **baixa e abre** o anexo, não sobre o servidor. Uma evolução natural, se necessário, é integrar um antivírus de código aberto (como o ClamAV) ao fluxo de upload.

7 Docker e Implantação (Deploy)

O projeto roda inteiramente em um único contêiner (`php:8.4-apache`), sem serviço de banco de dados. O `Dockerfile` instala as extensões PHP necessárias, copia o código, executa `composer install` e cria os diretórios de armazenamento com as permissões corretas. O `docker-compose.yml` expõe a porta 80 do contêiner (mapeada localmente para 8097) e monta dois volumes:

Listing 4: Volumes definidos em `docker-compose.yml`

```
1 volumes:
2   - ./storage/content:/var/www/html/storage/app/private/content
3   - ./storage/uploads:/var/www/html/storage/app/public/uploads
```

Esses dois diretórios são os **únicos dados reais do site** — todo o restante (código, tema) pode ser reconstruído a qualquer momento a partir do repositório Git. Qualquer rotina de backup do ambiente de produção deve necessariamente incluir essas duas pastas.

Para subir o ambiente localmente ou em produção:

```
1 docker compose up -d --build
```

8 Reaproveitando o Template para Outro Departamento

Como o objetivo original do projeto era servir de *modelo* reutilizável, o processo de adaptar este mesmo código para outro departamento é deliberadamente simples:

1. Copiar a pasta `site/` para o novo projeto.
2. Ajustar `APP_NAME`, `ADMIN_EMAIL` e gerar um novo `ADMIN_PASSWORD_HASH` no `.env`.
3. Subir o contêiner e preencher todo o conteúdo pelo painel `/admin` — nome, logos, textos, equipe, cursos, contato.

Não é necessário alterar nenhum arquivo PHP ou Blade para o uso básico: a estrutura de seções já cobre o que um site institucional de departamento tipicamente precisa.

9 Referências

- Framework Laravel — <https://laravel.com>
- Tema visual base "Neoton" (licença comercial, uso adaptado neste projeto)
- Identidade visual e logomarca: Universidade Federal de Ouro Preto (UFOP)